Vihaan Jain 251080026 IT

Task 2 : CPUSim - A Pipelined CPU & Memory Hierarchy Simulator

Index :

Github Link :- https://github.com/v1haan24/CPUSim-CacheAware-CPU-Scheduler

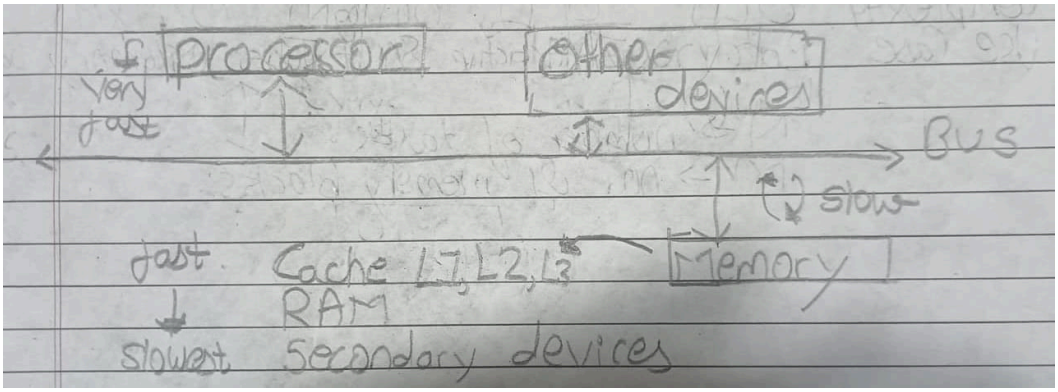
Introduction

Modern computer systems can execute millions of instructions every second, but processor speed has improved much faster than memory speed. Because of this gap, a CPU often has to wait for data to arrive from memory before it can continue execution. Even though the processor is capable of performing operations very quickly, memory access delays can significantly affect overall system performance.

To reduce this delay, computer systems use a cache hierarchy consisting of multiple cache levels placed between the CPU and main memory. Data found in cache can be accessed much faster than data fetched from RAM. At the same time, the operating system scheduler decides which task gets CPU time and in what order tasks are executed.

The concepts of processor memory speed difference, cache hierarchy and memory access latency were discussed in our **Computer Organization (CO)** in sem2, where we studied how memory bottlenecks can impact overall system performance despite having a fast CPU.

This creates an interesting challenge. A scheduling algorithm may focus on fairness and equal CPU allocation, while a memory aware scheduler may try to prioritize tasks whose data is already present in cache. Both approaches have advantages and trade offs. The main objective of this project is to simulate CPU scheduling and cache hierarchy together, observe how memory blocks move through different cache levels, and study how scheduling decisions influence cache performance, RAM accesses and total execution cost.



Fairness vs Performance

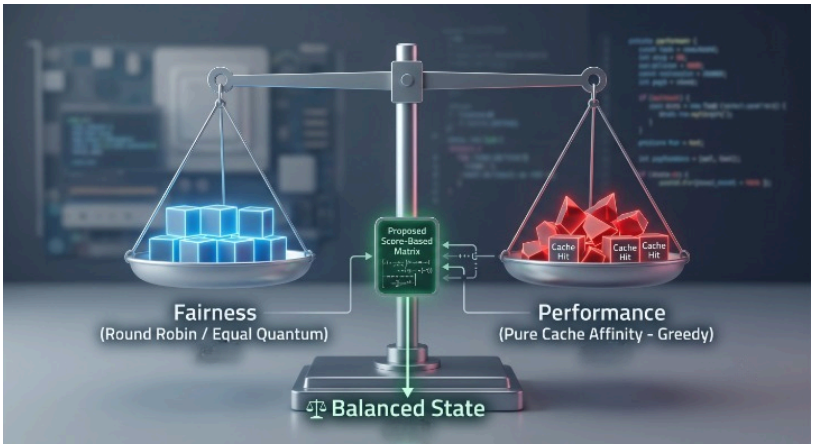
While designing the scheduler, the main question was what should be prioritized. One approach is to focus on fairness and ensure that all tasks receive CPU time regularly. Another approach is to focus on reducing memory access cost by selecting tasks whose required data is already available in the cache hierarchy.

Algorithms such as Round Robin are fair because every task gets an opportunity to execute. However, they do not consider where the required data is located. A task may be selected even when most of its memory blocks are not present in cache, resulting in additional cache misses and expensive RAM accesses.

While studying this behaviour, the idea came up that instead of selecting tasks purely based on arrival time or burst time, the scheduler could also look at the memory blocks required by each task. If a task is likely to use data that is already present in L1, L2 or L3 cache, executing it earlier could reduce memory access latency and improve overall performance.

However, selecting tasks only on the basis of cache locality can create another problem. Some tasks may continuously be delayed if their required blocks are not present in cache. This can reduce fairness and may even lead to starvation in certain cases.

Therefore, the main challenge in this project was to design a scheduling strategy that attempts to utilize existing cache contents whenever possible, while still ensuring that every task eventually gets an opportunity to execute.



Standard Algorithms

Before finalizing the scheduler design, several commonly used scheduling algorithms were studied and evaluated.

> First Come First Serve (FCFS) is one of the simplest scheduling algorithms where tasks are executed in the order they arrive. It is easy to implement and understand, but long tasks can delay shorter tasks, leading to higher waiting times.

> Round Robin (RR) improves fairness by allocating CPU time in fixed intervals called time quanta. Every task gets a chance to execute, making starvation almost impossible. However, Round Robin does not consider memory access patterns and may result in frequent context switching.

> Shortest Job First (SJF) prioritizes tasks with smaller burst times. This generally reduces average waiting time and improves throughput. The drawback is that longer tasks may wait for a long time if shorter tasks continue to arrive.

> Priority Scheduling assigns priorities to tasks and executes higher priority tasks first. While this can be useful in certain situations, low priority tasks may experience starvation.

While studying these approaches, it was observed that most traditional scheduling algorithms make decisions using factors such as arrival time, burst time or priority, but do not consider whether the data required by a task is already available in cache. Since memory access latency plays a major role in overall

execution cost, this observation motivated the development of a custom scheduling strategy that also considers cache availability while selecting the next task to execute.

The final scheduler used in this project was inspired by this idea and attempts to improve memory performance without completely ignoring fairness among tasks.

Proposed Solution

After evaluating the strengths and limitations of traditional scheduling algorithms, a custom scheduling strategy was developed for this project.

The idea behind this scheduler can be related to a simple daily life situation. Maan lo a student needs a notebook. If it is already lying on the study table, he will pick it up immediately. If it is kept in a nearby cupboard, a little extra effort is required. However, if it is stored somewhere deep inside another room, much more time is spent just reaching it. Naturally, an **aalsi** student would prefer using things that are already nearby instead of repeatedly walking to fetch them.

A similar observation was used while designing the scheduler. If the data required by a task is already available closer to the CPU, it makes sense to execute that task first rather than selecting a task whose data must be fetched from slower levels of memory.

Based on this idea, the scheduler examines the memory blocks associated with every active task before making a scheduling decision. A score is then calculated for each task depending on where its required memory blocks are currently located in the memory hierarchy. Blocks available in L1 cache contribute the highest score, followed by blocks present in L2 and L3 cache.

The task with the highest score is selected for execution. Since its required data is already available closer to the processor, memory access latency can be reduced and expensive RAM accesses can often be avoided.

At each scheduling step, all active tasks are evaluated, scores are calculated and the task with the highest score is selected for execution. After execution, the cache hierarchy is updated according to the memory block requested by the task, and the process repeats until all tasks are completed.

This approach attempts to combine scheduling and memory behaviour into a single decision making process, allowing the simulator to demonstrate how cache contents can influence overall system performance and memory access cost.

Drawbacks and solution

While the proposed scheduling strategy improves memory performance by preferring tasks whose required data is already available in cache, relying only on cache information can create a **fairness** problem. Tasks with lower cache scores may continuously get postponed if other tasks repeatedly have better cache availability.

To reduce this possibility, an aging mechanism was introduced into the scheduler. Every time a task is not selected for execution, its waiting counter is increased. This waiting time contributes a small bonus to the task's final score during future scheduling decisions.

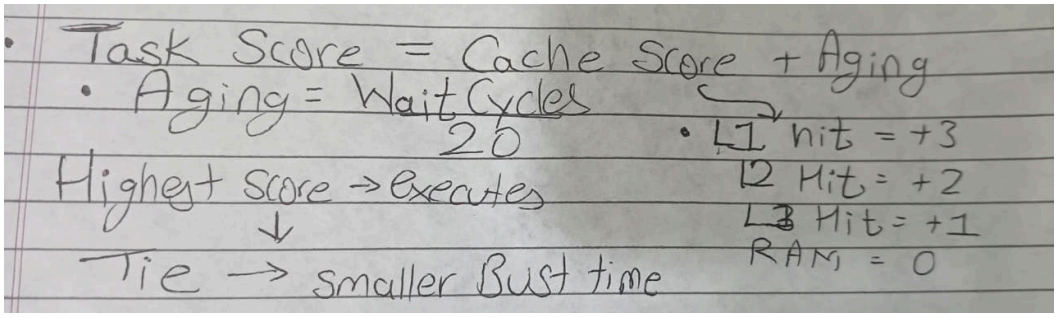
As a result, tasks that remain in the queue for a long time gradually become more likely to be selected, even if their cache score is relatively low. This helps prevent **starvation** while still allowing the scheduler to benefit from cache locality.

However, aging does not completely remove all fairness concerns. In certain situations, a **task with a very large burst time and strong cache locality** may continue receiving preference because of its high cache score. This can increase the waiting time of smaller tasks, although the aging mechanism and SJF tie breaker help reduce this effect over time.

In addition to the aging mechanism, a **shortest job first style tie breaker** is also used. When two tasks receive the same final score, preference is given to the

task with the smaller remaining burst time. This helps improve responsiveness and allows shorter tasks to complete earlier.

Therefore, the final scheduler does not rely only on cache information. Instead, it combines cache utilization, waiting time and task progress to achieve a better balance between performance and fairness.



Cache Hierarchy Design

The simulator implements a four level memory hierarchy consisting of L1 cache, L2 cache, L3 cache and RAM. Each cache level has a fixed capacity and a different access latency. As we move farther away from the CPU, storage capacity increases but access speed decreases.

Level	Capacity	Latency
L1	32 slots	4 cycles
L2	128 slots	12 cycles
L3	512 slots	40 cycles
RAM	unlimited	200 cycles

Whenever a task requests a memory block, the simulator searches for that block in the L1 cache first. If the block is not found, the search continues in L2 and then L3 cache. If the block is absent from all cache levels, it is fetched from RAM.

The simulator follows an inclusive cache design. This means that whenever a block is fetched from RAM, it is inserted into all cache levels so that future accesses can potentially be served from faster memory. The cache replacement policy used is First In First Out (FIFO), where the oldest block is removed whenever a cache level reaches its maximum capacity.

This hierarchy allows the simulator to clearly demonstrate cache hits, cache misses, evictions and RAM accesses, making the impact of memory locality visible during execution.

System Working

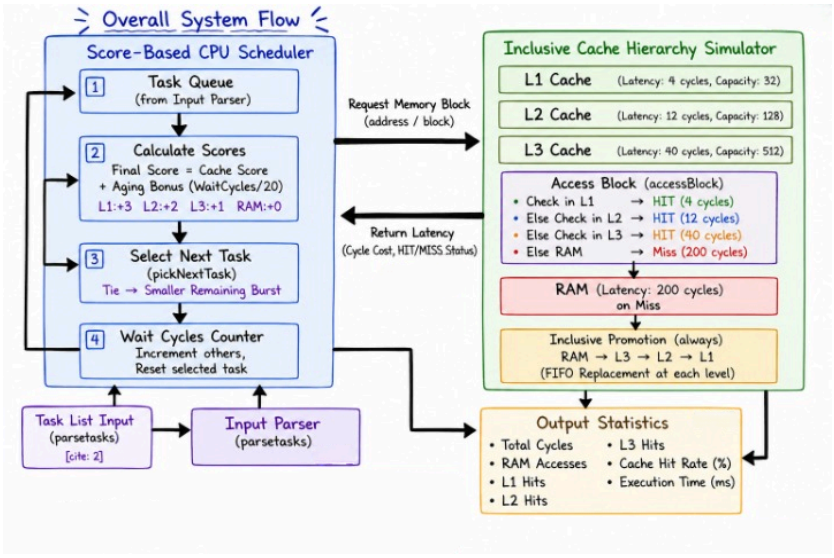
The simulator is divided into two major components: the scheduler and the cache hierarchy. The scheduler is responsible for selecting the next task to execute, while the cache hierarchy handles memory requests and determines whether data is obtained from L1, L2, L3 or RAM.

The execution begins by reading the input file and creating task objects containing task information such as task ID, burst time and memory block requirements. These tasks are then stored in the scheduler's task list.

During each simulation step, the scheduler evaluates all active tasks and calculates a score for each one based on cache availability and waiting time. The task with the highest score is selected for execution.

Once a task is selected, it requests its next memory block. The requested block is searched in the cache hierarchy starting from L1 cache, followed by L2 and L3 cache. If the block is not found in any cache level, it is fetched from RAM and promoted into the cache hierarchy according to the cache policy.

After the memory request is completed, cache statistics and task information are updated. This process continues until all tasks have finished execution.
This separation between scheduling logic and cache management makes the simulator easier to understand, modify and extend in the future



Implementation

The simulator was implemented in C++ and is divided into two main components: the scheduler and the cache hierarchy. Both components operate independently but interact during every execution step of the simulation.

Task Management

Each task is represented using a dedicated data structure containing information such as the task ID, burst time, remaining execution time and the list of memory blocks required by the task. Additional variables are maintained to track the next memory block that should be accessed, the amount of time a task has spent waiting and whether the task has completed execution.

When the simulator starts, all tasks are loaded from the input file and stored in memory. During execution, task information is continuously updated as burst times decrease and memory requests are processed.

Maintaining separate information for each task allows the scheduler to make decisions based not only on CPU requirements but also on memory behaviour and waiting time.

Cache Hierarchy Implementation

The memory subsystem consists of three cache levels and RAM. Each cache level is implemented as an independent structure containing its capacity, latency and the memory blocks currently stored within it.

The capacities used in the simulator are 32 blocks for L1 cache, 128 blocks for L2 cache and 512 blocks for L3 cache. RAM is assumed to have unlimited capacity and acts as the final source for all memory requests.

To support fast lookup operations, each cache level maintains a collection of memory blocks currently stored inside it. This allows the simulator to quickly determine whether a requested block is already available in cache.

A FIFO (First In First Out) replacement policy is used throughout the cache hierarchy. Whenever a cache reaches its maximum capacity, the oldest block is removed before a new block is inserted. This behaviour follows the requirements specified in the project statement and allows cache evictions to be visualized clearly during execution.

Memory Access Flow

Whenever a task executes, it requests a memory block from its memory access list. The simulator then searches for that block through the memory hierarchy in the following order:

L1 Cache → L2 Cache → L3 Cache → RAM

If the block is found in L1 cache, the request is completed immediately with the lowest latency cost. If the block is not present in L1, the search continues in L2 and then L3 cache.

When a block is found in L2 or L3 cache, the corresponding latency cost is applied and the block is promoted into higher cache levels so that future accesses can occur more quickly.

If the block is absent from all cache levels, a cache miss occurs and the block is fetched from RAM. Since RAM access is significantly slower than cache access, this operation incurs the highest latency cost in the simulation. After being fetched from RAM, the block is inserted into the cache hierarchy according to the simulator's cache policy.

This process closely models how modern computer systems search through progressively larger and slower memory levels before accessing main memory.

Scheduling Logic

The scheduler is responsible for deciding which task should receive CPU time during each execution step.

Instead of relying only on burst time or arrival order, the scheduler examines the memory requirements of every active task. For each task, a score is calculated based on the availability of its required memory blocks within the cache hierarchy.

Memory blocks present in L1 cache contribute the highest score because they can be accessed with the lowest latency. Blocks present in L2 and L3 cache contribute smaller scores because they require additional access time. Tasks whose memory blocks are already available closer to the CPU therefore receive higher overall scores.

The exact scoring values used by the scheduler are heuristic in nature and were chosen to preserve the relative importance of different cache levels. The objective was not to model latency values mathematically, but rather to ensure that memory blocks located closer to the CPU contribute more strongly to the scheduling decision than blocks located in lower cache levels. Different scoring schemes could be explored in future versions and may produce different scheduling behaviour depending on the workload.

The task with the highest score is selected for execution. This allows the scheduler to take advantage of cache locality and reduce expensive memory accesses whenever possible.

To reduce starvation, an aging mechanism is also included in the scheduling process. Every time a task waits without being selected, its waiting counter increases. This waiting time contributes a small bonus to the final scheduling score.

As a result, tasks that remain in the queue for extended periods gradually gain higher priority, preventing them from being ignored indefinitely.

In situations where two tasks receive the same final score, a Shortest Job First style tie breaker is applied. The task with the smaller remaining burst time is selected first. This helps improve responsiveness and allows shorter tasks to complete earlier.

The final scheduling decision therefore combines three different factors:

- Cache locality
- Waiting time
- Remaining burst time

This creates a balance between performance and fairness.

Statistics Collection

Throughout execution, the simulator continuously records performance statistics.

Separate counters are maintained for L1 cache hits, L2 cache hits, L3 cache hits and RAM accesses. These metrics provide insight into how effectively the scheduler utilizes the cache hierarchy.

The simulator also tracks total execution cycles. Every memory request contributes a latency cost based on the level from which the requested block was retrieved. These costs are accumulated to determine the total execution cost of the workload.

At the end of execution, a cache hit rate is calculated using the total number of successful cache accesses relative to all memory accesses.

These statistics allow different scheduling approaches to be compared objectively and provide a quantitative measure of memory performance.

Simulation Workflow

The complete execution process follows a repeating cycle until all tasks have finished execution.

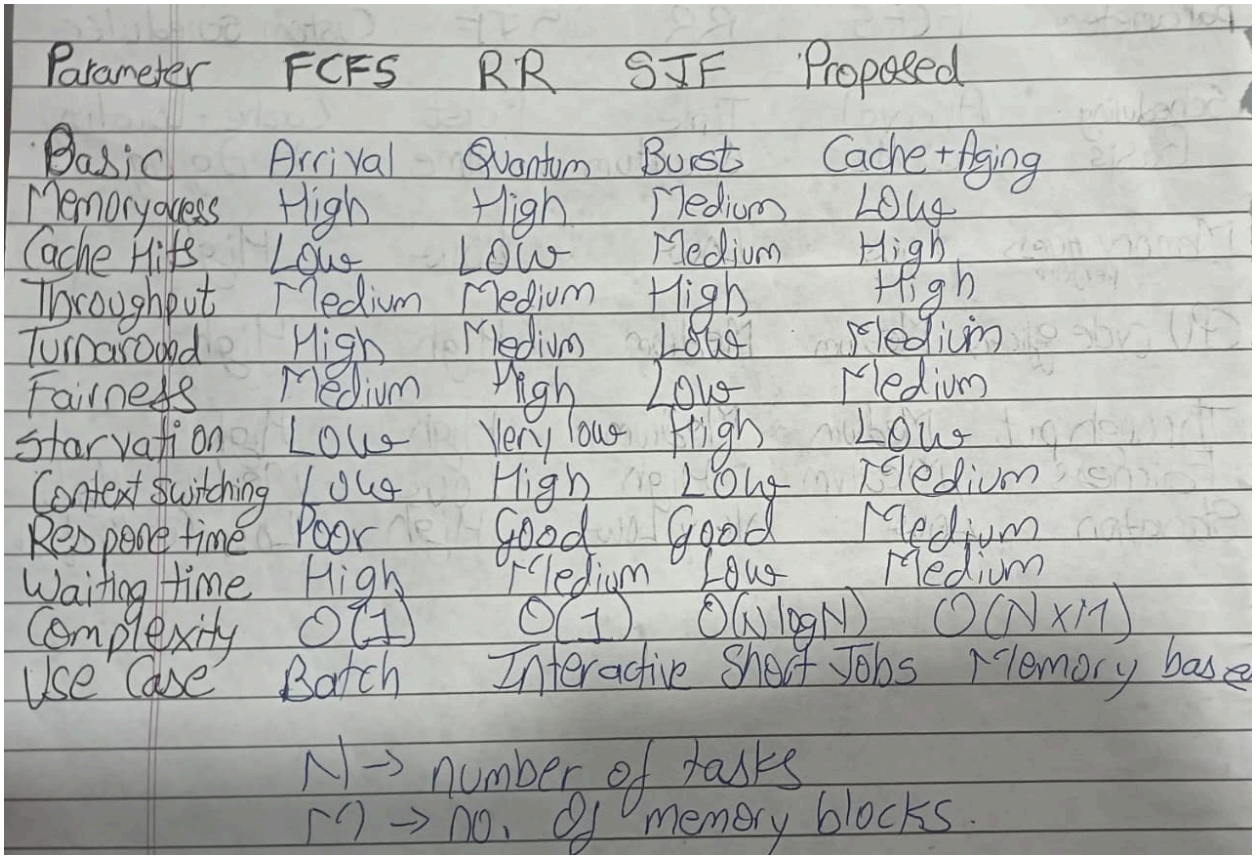
First, the scheduler evaluates all active tasks and calculates their scheduling scores. The highest scoring task is then selected for execution. The selected task requests its next memory block, which is searched through the cache hierarchy.

Based on the result of this search, cache structures and performance statistics are updated.

The task's remaining burst time is then reduced and completion status is checked. If the task has finished execution, it is removed from further scheduling decisions. Otherwise, it remains available for future scheduling cycles.

This process continues until every task has completed execution, after which final statistics and performance metrics are displayed.

Comparative Analysis



Parameter	FCFS	RR	SJF	Proposed
Basic	Arrival	Quantum	Burst	Cache+Aging
Memoryaress	High	High	Medium	LOw
Cache Hits	Low	LOw	Medium	High
Throughput	Medium	Medium	High	High
Turnaround	High	Medium	Low	Medium
Fairness	Medium	High	Low	Medium
starvation	Low	Very low	High	LOw
Context switching	Low	High	LOw	Medium
Response time	Poor	Good	Good	Medium
Waiting time	High	Medium	Low	Medium
Complexity	O(1)	O(1)	O(N log N)	O(N x M)
Use Case	Batch	Interactive	Short Jobs	Memory base

N -> number of tasks
M -> no. of memory blocks.

The proposed scheduler improves cache locality and reduces unnecessary RAM accesses, resulting in higher throughput, better memory performance, and fewer cache misses. The aging mechanism helps prevent starvation while maintaining fairness. Although the scheduler has O(N×M) complexity, modern CPUs execute scheduling decisions much faster than main memory accesses, so the additional overhead is often offset by reduced memory fetch delays. As a result, the scheduler is particularly effective for memory bound workloads.

Limitations

Although the proposed scheduler improves cache utilization and reduces expensive memory accesses, it is not without limitations. The scheduling decision is based on a heuristic scoring system and therefore may not always produce the optimal execution order for every workload. Some trade offs were intentionally accepted in order to balance memory performance and fairness.

- The scheduler evaluates all active tasks during each scheduling step, resulting in O(NxM) scheduling complexity. Still the CPU is very fast compared to fetching memory, so it compensates with cache hits.
- A task with very high cache affinity and a large burst time may continue receiving preference for long periods, increasing the waiting time of smaller tasks.
- Scheduling decisions are made using the current state of the cache hierarchy and do not consider future memory access patterns.
- The simulator assumes a single CPU core and therefore does not represent multicore scheduling behaviour.
- The cache model is simplified compared to real processors, which may use techniques such as cache associativity, hardware prefetching and cache coherence protocols.
- All tasks are loaded before execution begins. Dynamic task arrivals and real time workloads are not considered in the current implementation.

Another limitation is that the proposed scheduler evaluates all active tasks before making a scheduling decision. While this overhead is negligible for small and

medium workloads, it may become more noticeable as the number of tasks increases significantly. In such situations, the reduction in memory access cost must be large enough to justify the additional scheduling overhead. Therefore, the effectiveness of the approach depends on the characteristics and size of the workload being executed.

Despite these limitations, the simulator successfully demonstrates the interaction between CPU scheduling and memory hierarchy behaviour while maintaining a reasonable balance between performance and fairness.

Future Improvements

Although the current implementation successfully demonstrates the interaction between CPU scheduling and memory hierarchy, several improvements can be explored in future versions:

- Support for multicore processors can be added to better represent modern CPU architectures.
- Additional scheduling algorithms such as MLFQ, Priority Scheduling, and Weighted Round Robin can be implemented for comparison.
- More advanced cache replacement policies such as LRU (Least Recently Used) can be evaluated against FIFO.
- Dynamic task arrivals can be introduced instead of loading all tasks at the start of execution.
- The scheduling score can be made adaptive, allowing cache locality, waiting time, and burst time to be weighted differently for different workloads.
- A simple graphical user interface (GUI) can be developed to visualize task execution, cache hits, cache misses, and memory movement.
- Additional performance metrics such as turnaround time, waiting time, and throughput can be collected for deeper analysis.

These enhancements would make the simulator more realistic and enable a broader study of scheduling and memory management strategies.

Conclusion

> No system is perfect, and every design decision involves fundamental trade offs depending on the specific use case. This project explored the direct interaction between

CPU scheduling and cache hierarchy through a custom built simulator. Instead of relying solely on traditional CPU metrics, the proposed scheduler integrates data location within the memory hierarchy into its decision making.
> By prioritizing tasks with existing cache locality, the system successfully reduces memory access latency and maximizes cache utilization. To systematically balance performance with fairness and eliminate starvation, a soft aging mechanism and an SJF style tie breaker were seamlessly incorporated.
> Ultimately, the simulator clearly demonstrates how scheduling choices dictate memory behavior across cache hits, misses, promotions, and evictions. This project underscores that overall system performance is heavily driven by memory resource efficiency, providing practical validation for the core hardware trade offs studied in Computer Organization.